Casos de prueba cliente - servidor TicAmazon Isla 4

Preparación de las pruebas

Se probaron 20 casos de la interacción entre el cliente y el servidor y los 20 dieron el resultado esperado. Las salidas que aparecen son las reales de los programas.

Programas que participan:

- **cliente**: cliente de terminal, se comunica con el intermediario por HTTP en el puerto 8080.
- **intermediario**: atiende al cliente, guarda el carrito de cada sesión y consulta al servidor de productos.
- **servidorProductos**: atiende al intermediario con el protocolo interno de la isla por TCP en el puerto 9091 y lee y descuenta el stock en cafeteria.dat.

Antes de las pruebas se creó el inventario completo con el script, que deja 2 bodegas, 13 categorías y 104 productos:

```
rm -f cafeteria.dat
./cargarInventario.sh
./servidorProductos <ip>
./intermediario <ip> <ip>
./cliente <ip>
```

Los casos 1 al 18 se hicieron con el cliente, salvo el 15, que se hizo con curl. Los casos 19 y 20 se hicieron conectándose directo al servidor de productos con nc, para revisar sus respuestas del protocolo interno sin pasar por el intermediario. En las salidas se quitó la línea "Trying to connect to" que imprime VSocket en cada conexión.

Consultas de categorías y productos

Caso 1. Listar las categorías

Entrada: `CATEGORIAS`

Resultado esperado: las 13 categorías de las dos bodegas, cada una con su id y su nombre.

Resultado obtenido: correcto.

```
> CATEGORIAS
  cafes  Cafes
  bebfr  Bebidas frias
  bebca  Bebida caliente
  desay  Desayunos
  empar  Emparedados
  ensal  Ensaladas
  postr  Postres
  caram  Caramelos
  repos  Reposteria
  panes  Panes
  choco  Chocolates
  snack  Snacks
  helad  Helados
```

Caso 2. Productos de una categoría

Entrada: `ensal`

Resultado esperado: los 8 productos de Ensaladas con su precio y su stock.

Resultado obtenido: correcto.

```
> ensal
  Ensalada-Cesar | precio: 3800.00 | stock: 12
  Ensalada-Griega | precio: 3600.00 | stock: 12
  Ensalada-Frutas | precio: 2500.00 | stock: 15
  Ensalada-Atun | precio: 3700.00 | stock: 10
  Ensalada-Caprese | precio: 3900.00 | stock: 10
  Ensalada-Pollo | precio: 3800.00 | stock: 12
  Ensalada-Verde | precio: 2800.00 | stock: 15
  Ensalada-Pasta | precio: 3300.00 | stock: 12
```

Caso 3. Categoría que no existe

Entrada: `licor`

Resultado esperado: el servidor responde 22/0 y el cliente avisa que no hay productos.

Resultado obtenido: correcto.

```
> licor
  no hay productos disponibles en la categoria "licor"
```

Caso 4. Id de categoría con formato inválido

Entrada: `co` (solo 2 letras)

Resultado esperado: el intermediario rechaza la petición con error 400 sin consultar al servidor de productos.

Resultado obtenido: correcto.

```
> co
  error 400: La categoría debe tener entre 3 y 20 letras.
```

Carrito

Caso 5. Agregar con stock suficiente

Entrada: `agregar Ensalada-Caprese 2`

Resultado esperado: se reservan las 2 unidades y el servidor descuenta el stock en cafeteria.dat.

Resultado obtenido: correcto.

```
> agregar Ensalada-Caprese 2
  agregado 2 x Ensalada-Caprese @ 3900.00
```

Caso 6. Agregar un producto de la otra bodega

Entrada: `agregar Bombones 3`

Resultado esperado: Bombones está en Bodega-2 y Ensalada-Caprese en Bodega-1. El servidor lo encuentra igual, porque busca el producto en todas las bodegas del archivo.

Resultado obtenido: correcto.

```
> agregar Bombones 3
  agregado 3 x Bombones @ 2500.00
```

Caso 7. Agregar más de lo que hay (reserva parcial)

Entrada: `agregar Banana-Split 20`, con stock de 10

Resultado esperado: el servidor reserva las 10 que hay, responde 51 con cantidad 10, y el cliente avisa que fue parcial.

Resultado obtenido: correcto.

```
> agregar Banana-Split 20
  agregado 10 x Banana-Split @ 3000.00
  solo habia 10 disponibles de los 20 pedidos
```

Caso 8. Agregar sin stock

Entrada: `agregar Banana-Split 1`, después del caso 7

Resultado esperado: el servidor responde 51 con cantidad 0 y el cliente avisa que no hay stock. No se agrega nada al carrito.

Resultado obtenido: correcto.

```
> agregar Banana-Split 1
  no hay stock de Banana-Split
```

Caso 9. Agregar un producto que no existe

Entrada: `agregar Pizza-Hawaiana 1`

Resultado esperado: el servidor responde 27 y el cliente avisa que el producto no existe.

Resultado obtenido: correcto.

```
> agregar Pizza-Hawaiana 1
  Pizza-Hawaiana no existe
```

Caso 10. El stock queda descontado en el archivo

Entrada: `ensal`, después del caso 5

Resultado esperado: Ensalada-Caprese pasa de 10 a 8. La consulta vuelve a leer cafeteria.dat, así que el cambio confirma que la reserva quedó escrita en el archivo.

Resultado obtenido: correcto.

```
> ensal
  ...
  Ensalada-Caprese | precio: 3900.00 | stock: 8
  ...
```

Caso 11. Cantidad inválida

Entrada: `agregar Sundae 0`, `agregar Sundae abc` y `agregar Sundae` sin cantidad

Resultado esperado: las dos primeras se rechazan en el intermediario con error 400. La tercera la rechaza el propio cliente, porque no trae cantidad.

Resultado obtenido: correcto.

```
> agregar Sundae 0
  error 400: La cantidad debe estar entre 1 y 999.
> agregar Sundae abc
  error 400: La cantidad debe estar entre 1 y 999.
> agregar Sundae
  uso: AGREGAR <nombre_producto> <cantidad>
```

Factura y sesiones

Caso 12. Factura con el carrito vacío

Entrada: `factura`, antes de agregar productos

Resultado esperado: el intermediario responde que el carrito está vacío.

Resultado obtenido: correcto.

```
> factura
  el carrito esta vacio
```

Caso 13. Factura con productos

Entrada: `factura`, después de los casos 5 al 9

Resultado esperado: solo aparece lo que se reservó de verdad: 2 Ensalada-Caprese, 3 Bombones y 10 Banana-Split. El total es 7800 + 7500 + 30000 = 45300. Los casos 8 y 9 no agregan nada.

Resultado obtenido: correcto.

```
> factura
--- Factura TicAmazon ---
  Ensalada-Caprese x2 = 7800.00
  Bombones x3 = 7500.00
  Banana-Split x10 = 30000.00
TOTAL: 45300.00
```

Caso 14. Dos clientes al mismo tiempo

Entrada: dos clientes abiertos a la vez. Uno agrega `Sundae 2` y el otro `Cono-Doble 1`, y los dos piden la factura.

Resultado esperado: cada uno recibe su propio número de sesión y su factura trae solo lo que él agregó.

Resultado obtenido: correcto.

```
primer cliente:
  sesion 3
> agregar Sundae 2
  agregado 2 x Sundae @ 2200.00
> factura
  Sundae x2 = 4400.00
TOTAL: 4400.00

segundo cliente:
  sesion 4
> agregar Cono-Doble 1
  agregado 1 x Cono-Doble @ 1600.00
> factura
  Cono-Doble x1 = 1600.00
TOTAL: 1600.00
```

Caso 15. Sesión que no existe

Entrada: `curl "http://<ip>:8080/agregar?producto=Sundae&cantidad=1&sesion=99"`

Resultado esperado: el intermediario no conoce la sesión 99, así que responde con data-estado="sesion-invalida" y no reserva nada.

Resultado obtenido: correcto.

```
<div class="respuesta" data-estado="sesion-invalida" data-detalle="La sesión
no existe; pida una nueva en /sesion.">
```

Caso 16. El intermediario se reinicia con un cliente abierto

Entrada: el cliente agrega `Flan 1` en la sesión 7. Se cierra el intermediario, se vuelve a levantar y el cliente agrega `Flan 1` otra vez.

Resultado esperado: el intermediario nuevo no conoce la sesión 7. El cliente lo detecta, pide una sesión nueva y repite la petición una vez.

Resultado obtenido: correcto. El carrito de la sesión 7 se pierde, porque los carritos viven en la memoria del intermediario. El Flan que se había reservado en ella sigue descontado en cafeteria.dat.

```
> agregar Flan 1
  agregado 1 x Flan @ 1500.00
  (se reinicia el intermediario)
> agregar Flan 1
  la sesion ya no existe en el intermediario, se pide una nueva
  sesion 1
  agregado 1 x Flan @ 1500.00
> factura
  Flan x1 = 1500.00
TOTAL: 1500.00
```

Comunicación y respuestas del servidor de productos

Caso 17. Servidor de productos apagado

Entrada: se cierra servidorProductos y en el cliente se escribe `CATEGORIAS` y `agregar Sundae 1`.

Resultado esperado: el intermediario no logra conectarse, reintenta 2 veces y le avisa al cliente. El intermediario sigue funcionando.

Resultado obtenido: correcto.

```
> CATEGORIAS
  el servidor de productos no responde (2 reintentos)
> agregar Sundae 1
  el servidor de productos no responde (2 reintentos)
```

Caso 18. El servidor de productos vuelve a levantarse

Entrada: se vuelve a levantar servidorProductos, sin reiniciar el intermediario, y en el cliente se escribe `bebca`.

Resultado esperado: la consulta funciona otra vez, porque el intermediario abre una conexión nueva en cada mensaje.

Resultado obtenido: correcto.

```
> bebca
  CafeCaliente | precio: 1400.00 | stock: 30
  Chocolate | precio: 1600.00 | stock: 25
  TeVerde | precio: 1100.00 | stock: 30
  TeNegro | precio: 1100.00 | stock: 30
  Manzanilla | precio: 1000.00 | stock: 20
  AguaDulce | precio: 900.00 | stock: 20
  ChaiLatte | precio: 1900.00 | stock: 15
  Matcha | precio: 2100.00 | stock: 15
```

Caso 19. Mensajes válidos del protocolo interno

Entrada: cada mensaje se envía con `printf "<mensaje>\n" | nc <ip> 9091`.

Resultado esperado:

- 10 se responde con 11 y la lista `id:nombre`.
- 20 se responde con 22, la cantidad y la lista, o con 22/0 si la categoría no existe.
- 50 se responde con 51 y lo reservado, o con 27 si el producto no existe.

Resultado obtenido: correcto.

```
>> INT_04|SERV/10
SERV|INT_04/11/cafes:Cafes,bebfr:Bebidas frias,bebca:Bebida
caliente,desay:Desayunos,empar:Emparedados,ensal:Ensaladas,postr:Postres,cara
m:Caramelos,repos:Reposteria,panes:Panes,choco:Chocolates,snack:Snacks,helad:
Helados
>> INT_04|SERV/20/postr
SERV|INT_04/22/8/Tiramisu,2500.00,10;Cheesecake,2600.00,10;Flan,1500.00,15;Br
ownie,1200.00,20;TresLeches,2000.00,12;Pie-
Limon,2200.00,10;Mousse,2100.00,10;Arroz-con-Leche,1400.00,15
>> INT_04|SERV/20/licor
SERV|INT_04/22/0
>> INT_04|SERV/50/Sundae/3
SERV|INT_04/51/Sundae/2200.00/3
>> INT_04|SERV/50/Pizza/1
SERV|INT_04/27/Pizza
```

La respuesta al 10 mide 221 caracteres y la del 20 mide 181. Si se pidiera una categoría con más productos, pasaría de 256 sin problema, porque el protocolo ya no tiene ese límite.

Caso 20. Mensajes inválidos del protocolo interno

Entrada: un mensaje con origen de cliente (`CLI_04`), uno con un tipo que no existe (`99`) y uno sin formato (`hola`).

Resultado esperado: el servidor responde 90 con el tipo original, el campo que falla y el valor recibido. Solo atiende mensajes que vienen de un intermediario.

Resultado obtenido: correcto.

```
>> CLI_04|SERV/10
SERV|CLI_04/90/10/origen/CLI_04
>> INT_04|SERV/99
SERV|INT_04/90/99/tipo_mensaje/99
>> hola
SERV|INT_04/90/00/mensaje/invalido
```

Resumen

Los 20 casos dieron el resultado esperado.

- **Consultas (casos 1 al 4):** categorías con nombre, productos, categoría inexistente e id mal formado.
- **Carrito (casos 5 al 11):** reserva completa, producto de la otra bodega, reserva parcial, sin stock, producto inexistente, stock descontado en el archivo y cantidad inválida.
- **Factura y sesiones (casos 12 al 16):** factura vacía, factura con total correcto, dos clientes con carritos separados, sesión inválida y renovación de la sesión.
- **Comunicación y protocolo (casos 17 al 20):** servidor de productos apagado, recuperación sin reiniciar el intermediario, mensajes válidos y mensajes inválidos del protocolo interno.

Comportamiento a tomar en cuenta: si el intermediario se reinicia, los carritos abiertos se pierden, pero lo que ya se había reservado sigue descontado en cafeteria.dat (caso 16). Para volver al inventario completo hay que crear el archivo de nuevo con cargarInventario.sh.